DOCUMENTACION DE SISTEMA

RootCause Bitcoin Defense

11 · Seguridad

Version analizada 0.3.0 · commit fd59bca · generado el 2026-09-07
Documento 11 de la serie · fuente: docs/system-documentation/11-security.md

Contenido

1 · Modelo de seguridad en una frase
2 · Autenticación y autorización
3 · Protección CSRF
4 · Validación y saneamiento de entrada
5 · El rechazo de material secreto
6 · Cifrado
7 · Gestión de secretos
8 · Registro y auditoría
9 · Cabeceras de seguridad
10 · Superficie de ataque
11 · Dependencias y cadena de suministro
12 · Resultado de los controles ejecutados
13 · Controles ausentes o no comprobados
14 · Buenas prácticas destacables
15 · Comunicación de vulnerabilidades

Superficie de ataque, controles implementados, controles ausentes y qué se verifica automáticamente en cada build.

> Complementa a `../THREAT_MODEL.md`, que modela adversarios y activos, y a `SECURITY.md`, que es la política pública. Aquí se hace el **análisis del código**: qué control existe, en qué línea, y qué no cubre. **Este documento no reporta ningún secreto encontrado.** El escaneo del histórico se ejecutó y su resultado está en la sección 12.

---

1 · Modelo de seguridad en una frase

El sistema asume **un único operador con acceso físico a su máquina**, no implementa autenticación, y sustituye ese control por el bind en loopback más una guardia de origen. Todo lo demás se deriva de ahí.

Consecuencia	Detalle
Quien alcanza el puerto, manda	No hay usuarios ni permisos que aplicar
Exponer el puerto invalida el modelo	Por eso `BTC-OPSEC-001` es crítico
El sistema se protege de sí mismo	Rechaza secretos, no puede firmar, no puede gastar

---

2 · Autenticación y autorización

Control	Estado
Autenticación de usuario	[FALLO] **No existe**
Autorización por rol	[FALLO] No existe. No hay roles
Gestión de sesiones	[FALLO] No existe. **No se emite ni se lee ninguna cookie**
Tokens o claves de API	[FALLO] No existen
Identidad del actor	[ATENCION] Etiqueta `x-rootcause-actor`, **sin verificar**

`actorFrom()` valida la forma de la etiqueta (`/^[a-z0-9._@-]{1,80}$/i`) y cae a `local-user` si no encaja. **Sirve para atribuir en el registro de auditoría, no para autenticar.** Cualquiera que pueda hacer la petición puede firmar con cualquier etiqueta.

Qué sustituye a la autenticación, en orden de importancia:

#	Control	Archivo
1	Bind en `127.0.0.1` por defecto	`src/config.js`
2	`x-rootcause-request: 1` obligatoria en toda mutación	`src/app.js`
3	Rechazo de `sec-fetch-site` cross-site	`src/app.js`
4	Ausencia total de cabeceras CORS	Por omisión
5	`BTC-OPSEC-001` marca crítico salir de loopback	`src/domain/posture.js`

---

3 · Protección CSRF

Es el control más importante del sistema, porque el navegador del operador puede visitar cualquier sitio mientras el panel está abierto.

Vector	¿Cubierto?	Cómo
Formulario HTML de otro sitio hacia el panel	[OK] Sí	Un formulario no puede fijar `x-rootcause-request`, y una cabecera personalizada obliga a una comprobación previa CORS que el servidor no responde
`fetch` cross-origin con cabecera personalizada	[OK] Sí	La comprobación previa falla: no hay `access-control-allow-headers`
Petición con `sec-fetch-site: cross-site`	[OK] Sí	Rechazo explícito con `403`
Cliente que no es navegador, desde la misma máquina	[FALLO] **No cubierto**	Puede poner cualquier cabecera. La defensa aquí es que ya está dentro de la máquina
Petición sin `sec-fetch-site`	[ATENCION] **Pasa** el segundo control	Los navegadores actuales la envían siempre; queda el primer control

> El diseño es correcto para su modelo de amenazas: **contra una página web hostil, funciona; contra un proceso local hostil, no pretende funcionar** —si hay un proceso hostil en la máquina, el estado cifrado y su clave están ya al alcance de problemas mayores.

4 · Validación y saneamiento de entrada

4.1 · Capas de validación, en orden

#	Capa	Rechaza
1	`content-type`	Cualquier cuerpo que no sea JSON
2	Límite por fragmento	Cuerpos por encima de 64 KiB, **durante** la lectura
3	`JSON.parse`	JSON malformado
4	`assertNoSecretMaterial`	Material secreto, a cualquier profundidad
5	Validadores por campo	Longitudes, enums, hex, enteros seguros
6	Validación cruzada	Coherencia de quorum y firmantes
7	Invariantes de negocio	Huella duplicada

4.2 · Validadores por tipo

Tipo	Regla	Ejemplo de lo que rechaza
Huella	`/^[0-9A-F]{8}$/` tras `trim` y mayúsculas	`"A1B2C3"`, `"ZZZZZZZZ"`
Hash de transacción	`/^[0-9a-f]{64}$/` tras `trim` y minúsculas	Cualquier longitud distinta
Texto	No vacío tras `trim`, longitud máxima	Cadenas de 10 000 caracteres
Entero	`Number.isSafeInteger` + rango	`1e400`, `"3.5"`, `-1`
Enum	Pertenencia a un `Set`	`"multisig-2of3"`
Identificador de incidente en la ruta	`/^[a-f0-9]{20}$/`	Cualquier intento de inyección en la ruta

Rutas de archivo: ninguna se construye con datos del usuario. `STATIC_FILES` es una tabla de rutas fijas, y el `path.join` usa el nombre de esa tabla. **Sin concatenación de entrada arbitraria, no hay path traversal.**

4.3 · Saneamiento de salida

Superficie	Control	Archivo
Todo dato en el DOM	`escapeHtml` sobre `& < > " '`	`src/web/static/app.js`
URLs de fuentes de avisos	`safeUrl`, solo `http:` y `https:`	Ídem
Metadata de auditoría	`redactForAudit` antes de escribir	`src/infrastructure/audit-log.js`
Mensajes de error 5xx	Sustituidos por un texto genérico	`src/app.js`
Etiqueta de endpoint RPC	`endpointLabel()` descarta credenciales y ruta	`src/infrastructure/bitcoin-rpc.js`
Evidencia de secretos en el informe	Enmascarada: 3 caracteres, …, 2 caracteres	`scripts/check-history-secrets.js`

> El último es un detalle que se pasa por alto a menudo: **un informe de secretos no debe convertirse él mismo en una filtración.**

5 · El rechazo de material secreto

El invariante que define el producto. Comprobado con cinco cargas hostiles en cada build por `scripts/check-security-claims.js`:

Carga de prueba	Vector	Resultado exigido
`{ seed: "abandon abandon abandon" }`	Campo de primer nivel	`422`
`{ signers: [{ mnemonic: "..." }] }`	Campo anidado en array	`422`
`{ metadata: { recovery_phrase: "..." } }`	Objeto anidado, nombre con guion bajo	`422`
`{ passphrase: "correct horse battery staple" }`	Passphrase BIP-39	`422`
`{ notes: "xprv..." }`	**Clave extendida en un campo de nombre inocente**	`422`

Y una sexta que debe **pasar**: una wallet con procedencia pública completa debe devolver `201`. El guardia protege sin bloquear el uso legítimo.

Cobertura	Estado
Nombre de campo, con normalización de estilo	[OK]
Valor con forma `xprv`/`tprv`	[OK]
Valor con forma WIF	[OK]
Profundidad arbitraria en objetos y arrays	[OK]
Valor codificado en base64 que oculte una semilla	[FALLO] **No cubierto** — limitación aceptada
Mnemónico de 12 palabras en un campo de nombre inocente	[FALLO] **No cubierto en la API** — sí en el escáner del histórico

> Los dos huecos son deliberados: el objetivo es **impedir el error y el descuido**, no derrotar a un usuario decidido a autolesionarse. Aun así son huecos, y están registrados en `15-risks-and-technical-debt.md`.

6 · Cifrado

Aspecto	Implementación	Valoración
Algoritmo	AES-256-GCM	[OK] AEAD, autentica además de cifrar
IV	12 bytes de `crypto.randomBytes` **por escritura**	[OK] Correcto; nunca se reutiliza
Tag de autenticación	Verificado en el descifrado	[OK] Un byte alterado hace fallar
Longitud de clave	32 bytes exactos, exigidos	[OK]
Derivación de clave	**Ninguna.** Se usa tal cual	[ATENCION] Correcto para una clave generada; arriesgado para una escrita a mano

Aspecto	Implementación	Valoración
Detección de clave débil	`estimateKeyStrengthBits` → `BTC-OPSEC-003`	[OK] Mitiga lo anterior
Rotación de clave	**No existe**	[ATENCION] Cambiarla obliga a descifrar y volver a cifrar a mano
Cifrado en tránsito	**No hay TLS**	[OK] Correcto: es loopback
Metadatos en claro	Solo el sobre; **ningún dato del estado**	[OK] Verificado por prueba

Fuentes de aleatoriedad: `crypto.randomBytes` para la clave y los IV, `crypto.randomUUID` para los identificadores, `crypto.getRandomValues` para el txid del simulador. Todas criptográficas. **No hay un solo `Math.random()` en el código de producción.**

---

7 · Gestión de secretos

Secreto	Dónde vive	Dónde nunca debe estar
`ROOTCAUSE_DATA_KEY`	Entorno del proceso, o `config.cmd` del perfil	Repositorio, carpeta del programa, junto al archivo de datos
Credenciales RPC	Entorno, o preferiblemente la cookie del nodo	La URL de RPC — `validateRpcUrl` **lanza** si aparecen ahí
Cookie de Bitcoin Core	Archivo del nodo, **releída en cada llamada**	Copiada a ningún sitio

Control	Dónde
`.env` y `data/*` ignorados por git	`.gitignore`
Detección de PEM y `xprv` en todo archivo de texto	`scripts/validate-repo.js`
11 patrones de secreto sobre todo el histórico	`scripts/check-history-secrets.js`
Rutas prohibidas: `.env`, `.cookie`, `id_rsa`, `id_ed25519`, `.pfx`, `.p12`, `.keystore`, `data/state.enc.json`	Ídem
`gitleaks` verificado por checksum	`security-audit.yml`
El paquete rechaza incluir `data`, `.env`, `.git` o `node_modules`	`build-portable.ps1`
Exenciones del escáner, declaradas y contadas	`config/secret-scan-allow.json`

> Las exenciones no son silenciosas: el informe **cuenta las usadas y denuncia las que ya no hacen falta**, porque una exención de más tapa hallazgos futuros.

---

8 · Registro y auditoría

Propiedad	Valor
Qué se registra	9 tipos de acción, con actor, entidad y metadata

Propiedad	Valor
Integridad	Cadena de hashes SHA-256 sobre forma canónica
Verificación	`verifyAuditChain()`, expuesta en `GET /api/audit` y en `summary`
Reacción a la manipulación	`BTC-OPSEC-008`, **crítico**
Redacción	`redactForAudit` **antes** de calcular el hash
Inmutabilidad	Ninguna operación edita ni borra entradas
Retención	Indefinida. El registro solo crece

Detecta	No detecta
[OK] Una entrada editada	[FALLO] Reescritura completa del archivo con hashes recalculados
[OK] Una entrada eliminada	[FALLO] Acciones hechas fuera de la aplicación
[OK] Entradas reordenadas	[FALLO] Quién fue realmente: el actor es una etiqueta sin verificar

Logging operativo: líneas JSON en `stdout` y `stderr` para arranque, errores 5xx y fallos del watchtower. Cada respuesta lleva un `x-request-id`, aunque **ese identificador no aparece en el log del error**, así que hoy no se pueden correlacionar. Anotado como deuda.

---

9 · Cabeceras de seguridad

Ya detalladas en `06-deep-code-explanation.md`. Resumen de cobertura:

Amenaza	Cabecera	Estado
XSS por script externo	`script-src 'self'`	[OK]
XSS inline	Sin `unsafe-inline`	[OK]
Exfiltración a otro origen	`connect-src 'self'`	[OK]
Clickjacking	`frame-ancestors 'none'` + `x-frame-options: DENY`	[OK] Doble
Inyección de `<base>`	`base-uri 'self'`	[OK]
Envío de formulario a otro sitio	`form-action 'self'`	[OK]
Plugins y objetos	`object-src 'none'`	[OK]
Confusión de MIME	`nosniff`	[OK]
Fuga de la URL local	`no-referrer`	[OK]
Acceso a hardware	`permissions-policy`	[OK]
Aislamiento entre ventanas	COOP y CORP	[OK]

`scripts/check-local-only.js` verifica en cada build que la CSP conserva sus cinco directivas clave y que no admite `http:`, `*` ni `unsafe-inline`.

10 · Superficie de ataque

Superficie	Exposición por defecto	Controles
Puerto 8787	**Solo loopback**	Bind, límite de tasa, guardia de mutación
15 endpoints de API	Loopback	Validación en 7 capas
7 rutas estáticas	Loopback	Allowlist; sin construcción de rutas
Salida RPC hacia Bitcoin Core	Solo si se configura	Allowlist de 14 métodos, loopback, sin redirecciones, 2 MiB, timeout
Archivo de estado en disco	Perfil del usuario	AES-256-GCM, `0600`, escritura atómica
Panel en el navegador	Local	CSP, escape en todo render
Cadena de suministro	**Cero dependencias npm**	Verificado en cada build
Runtime Node embebido en el paquete	Sí	SHA-256 contra `SHASUMS256.txt`; auditado contra el Security WG
Workflows de CI	Repositorio	Actions pinneadas a SHA, permisos mínimos, `zizmor`

10.1 · Lo que no puede hacer el sistema, por construcción

Capacidad ausente	Cómo se garantiza
Firmar una transacción	No hay código de firma; ningún método RPC de firma en la allowlist
Transmitir una transacción	Ídem; `sendrawtransaction` no está y CI lo comprueba con `grep`
Almacenar una clave privada	El guardia rechaza en la frontera
Ejecutar código de terceros	Cero dependencias
Salir a Internet en uso normal	CSP `connect-src 'self'`; sin destinos remotos en el panel
Escalar privilegios en el contenedor	`cap_drop: ALL`, `no-new-privileges`, `USER 1000`

11 · Dependencias y cadena de suministro

Capa	Estado
Dependencias npm	**Cero.** Un scan de CVE sobre `node_modules` no tendría nada que mirar
Runtime distribuido	[OK] Auditado contra el Node.js Security WG, semanalmente
Actions de GitHub	[OK] Pinneadas a SHA; Dependabot semanal, agrupado
Binarios descargados en CI	[OK] `gitleaks` verificado por checksum oficial
`zizmor` y `hadolint`	[ATENCION] Pinneados por versión, no por hash

Capa	Estado
Imagen base Docker	[ATENCION] `node:24-alpine` por etiqueta, no por digest

> **El detalle honesto:** decir «cero vulnerabilidades» sobre un repositorio sin dependencias sería verdad y sería engañoso. El código ajeno que **sí** se distribuye es el motor Node dentro del paquete de Windows, y eso es exactamente lo que audita `check-runtime-advisories.js` — declarando siempre su cobertura, y saliendo con error si no hay red en vez de fingir un verde.

---

12 · Resultado de los controles ejecutados

Ejecutados sobre el commit `acdf4dd` durante este análisis:

Control	Resultado
`node --test`	[OK] 69 de 69
`node scripts/validate-repo.js`	[OK] 156 archivos, sin material privado embebido
`node scripts/check-local-only.js`	[OK] Cero dependencias, cero orígenes remotos, CSP intacta
`node scripts/check-security-claims.js`	[OK] **20 invariantes verificados contra la aplicación en marcha**
`node scripts/check-docs.js`	[OK] Todas las referencias resuelven
`node scripts/check-doc-style.js`	[OK] Sin problemas de estilo
`node scripts/generate-rule-docs.js --check`	[OK] Fichas sincronizadas
`node scripts/check-history-secrets.js`	[OK] **Sin material secreto en el histórico** — ver 12.1
`node scripts/check-runtime-advisories.js`	[OK] **Runtime distribuido sin avisos aplicables** — ver 12.2
CodeQL `security-extended`	[NO COMPROBADO] **No ejecutado**: corre en GitHub
`gitleaks`, `zizmor`, `hadolint`	[NO COMPROBADO] **No ejecutados**: corren en CI

> Las dos últimas filas son [NO COMPROBADO], **no verdes**. Una comprobación que no se hizo no es una comprobación superada. Es la misma regla que el propio producto aplica a sus informes.

12.1 · Secretos en el histórico de git

Campo	Resultado
Alcance	Histórico completo, `git log --all -p`
Binarios	**Revisados solo por su ruta, no por su contenido** — ver el aviso de abajo
Commits revisados	9
Líneas añadidas revisadas	20 207
Patrones aplicados	11
Cobertura declarada	100 % de las **líneas de texto** añadidas visibles en git

Campo	Resultado
Hallazgos	**Ninguno**
Exenciones en línea	2, marcadas con `secret-scan: allow`
Exenciones declaradas usadas	1 — la wordlist BIP-39 del propio detector, 30 coincidencias
Exenciones obsoletas	Ninguna
No cubre	Objetos colgantes ni ramas ya borradas del remoto

> La única exención declarada sigue justificada: sin la muestra de wordlist BIP-39, el detector no reconocería un mnemónico real, y con ella se detecta a sí mismo. El informe la cuenta, que es lo que impide que una exención se vuelva invisible.

Por qué los binarios quedan fuera del escaneo de contenido. Un escáner de secretos busca patrones de texto. Aplicado a los bytes de un PDF, una imagen o un ZIP, encuentra coincidencias por azar: un stream comprimido acaba conteniendo, antes o después, algo con la forma de `esquema://usuario:clave@`. Ocurrió de verdad con los PDF de esta documentación, y tumbó el trabajo de auditoría de CI con un hallazgo falso.

Decisión	Motivo
El **contenido** de un binario no se inspecciona	Un detector que grita cada vez que se regenera un PDF enseña a ignorarlo
Su **ruta** sí se comprueba	Un `.p12` o un `.pfx` no deben estar en el repositorio aunque nadie lea sus bytes
El informe declara cuántos se omitieron	La cobertura se declara siempre, y eso incluye lo que se deja fuera a propósito
`.gitattributes` marca `*.pdf binary`	Ataja la causa: sin declararlo, `text=auto` deja a una heurística si git vuelca los bytes en el diff

> Es la aplicación al propio detector de la regla que gobierna el proyecto: un falso positivo recurrente no es un control estricto, es un control que la gente aprende a saltarse.

12.2 · Cadena de suministro del runtime

Campo	Resultado
Base consultada	Node.js Security Working Group, 194 avisos evaluados
Runtime **distribuido**	`v22.23.2`, leído de `BUILD-INFO.json` del paquete ensamblado
Avisos aplicables al runtime distribuido	**Ninguno**
Runtime que ejecutó el análisis	`v24.11.1` — 36 avisos, todos **informativos**
Cobertura declarada	2 versiones contrastadas; 0 dependencias npm que auditar

> La distinción importa: los 36 avisos afectan al Node **local** de quien ejecutó la comprobación, no al que viaja dentro del artefacto. El script los marca como informativos y no bloquea, porque teñir de rojo por el Node del runner escondería el dato que sí importa. **El que se distribuye está limpio.** Recomendación aparte, y menor: actualizar el Node local a `^24.18.1` cuando se pueda.

13 · Controles ausentes o no comprobados

#	Control ausente	Severidad	Justificación del proyecto
1	Autenticación	[ATENCION] Media	Modelo de un solo operador local, declarado
2	TLS	[OK] No aplica	Loopback
3	Bloqueo tras intentos fallidos	[OK] No aplica	No hay credenciales
4	Rotación de la clave de datos	[ATENCION] Media	Se puede hacer a mano
5	`fsync` explícito tras escribir	[ATENCION] Baja	El `rename` cubre el caso realista
6	Purga del mapa del limitador	[ATENCION] Baja	Una entrada en el despliegue soportado
7	Comprobación de `schemaVersion` al cargar	[ATENCION] Baja	Solo existe una versión
8	Caducidad efectiva de aprobaciones	[ATENCION] Media	`expiresAt` se guarda pero no se comprueba
9	Integridad referencial de `walletId`	[ATENCION] Media	Un evento puede citar una wallet inexistente
10	Firma del artefacto publicado	[ATENCION] Media	Se publican checksums; **no hay firma criptográfica**
11	Digest fijo de la imagen base Docker	[ATENCION] Baja	Etiqueta `node:24-alpine`
12	Correlación de `x-request-id` con el log	[ATENCION] Baja	Se emite, no se registra

Clasificación completa con impacto y probabilidad en `15-risks-and-technical-debt.md`.

---

14 · Buenas prácticas destacables

Cosas que este repositorio hace y que no son habituales:

Práctica	Dónde
El claim de seguridad se **ejecuta**, no se afirma	`check-security-claims.js` arranca la app real y la ataca
El propio producto se audita a sí mismo	Familia `BTC-OPSEC`
La antigüedad de la inteligencia es un hallazgo	`BTC-INTEL-001`
Una etapa no ejecutable sale [NO COMPROBADO], nunca [OK]	Todo el criterio de auditoría
El informe de secretos enmascara su propia evidencia	`check-history-secrets.js`
Las exenciones se cuentan y las obsoletas se denuncian	Ídem
El artefacto se **arranca** antes de publicarse	`build-portable.ps1` y `release-windows.yml`
Se verifica que la app instalada tenga contenido	`summary.totals.wallets >= 1` en el release
El escáner de secretos revisa **todo el histórico**, no el árbol	`git log --all -p`

Práctica	Dónde
Las supresiones de `zizmor` llevan el motivo escrito al lado	`.github/zizmor.yml`

---

15 · Comunicación de vulnerabilidades

El procedimiento está en `SECURITY.md`: canal de contacto, qué información incluir y compromiso de respuesta del mantenedor.

Este análisis no realizó ninguna prueba destructiva, ni contra sistemas externos, ni de fuerza bruta, ni de denegación de servicio. Se limitó a leer el código y a ejecutar las comprobaciones que el propio repositorio provee.

Siguiente	
Pruebas y calidad	`12-testing-and-quality.md`
Riesgos clasificados	`15-risks-and-technical-debt.md`
Volver al índice	`README.md`